



# Remote procedure call (RPC)



Reti di Calcolatori Modulo 2

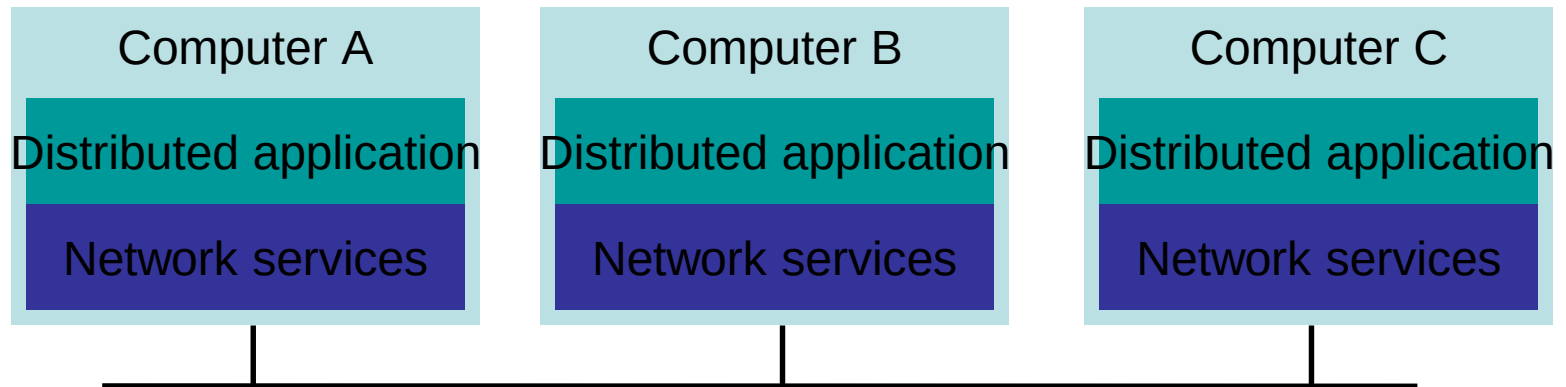
# Motivation

- ▶ Implementing a distributed application on top of sockets is tedious
  - ▶ Dealing with challenges of distributed systems on your own
  - ▶ Distribution transparency
  - ▶ Interoperability (e.g., heterogeneity data representation)
  - ▶ Security
  - ▶ Common services: Naming, Persistency, Events, Transactions, ...
- ▶ Higher level of abstraction: middleware
  - ▶ Deals with challenges of distributed systems (to a certain degree)
  - ▶ Provide additional services (e.g., naming, persistency,...)

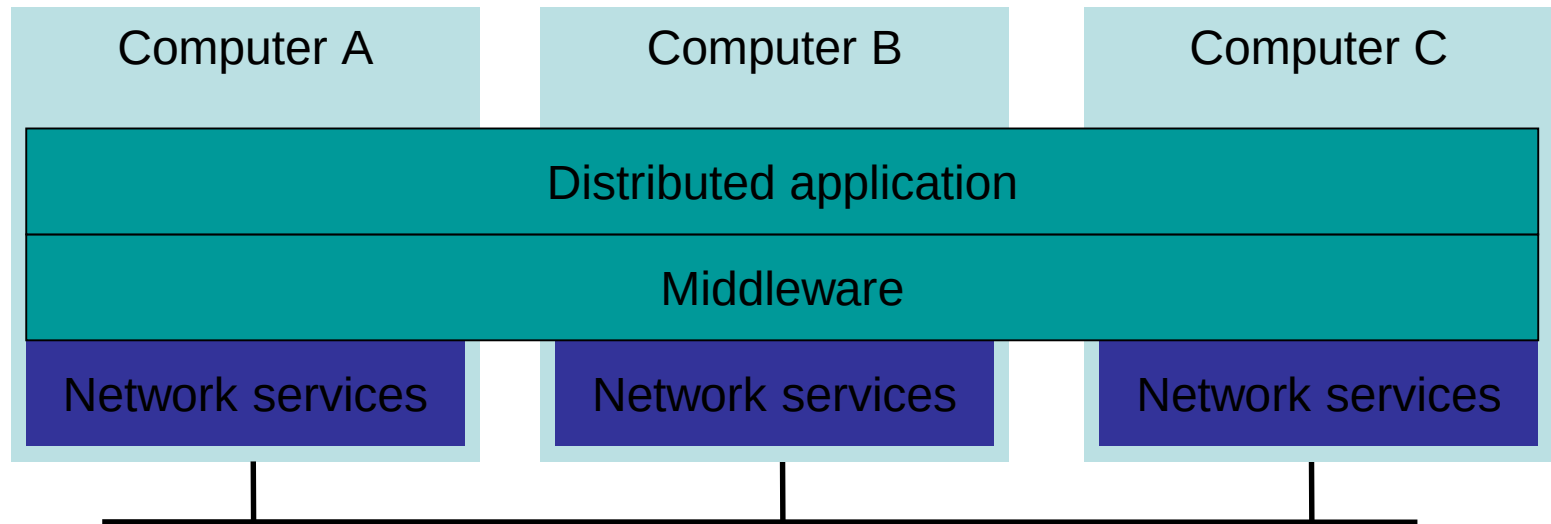
# Definition(s)

- ▶ “There is no good definition for middleware”
- ▶ “The slash between client/server”
- ▶ “The intersection of the stuff that network engineers don’t want to do with the stuff that application developers don’t want to do”

# “Classical” approach



# Middleware

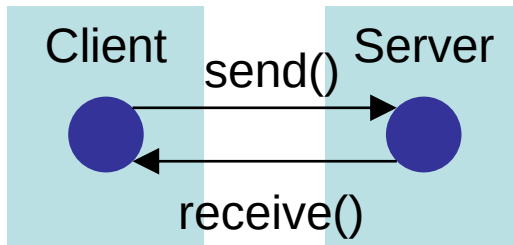


# Services

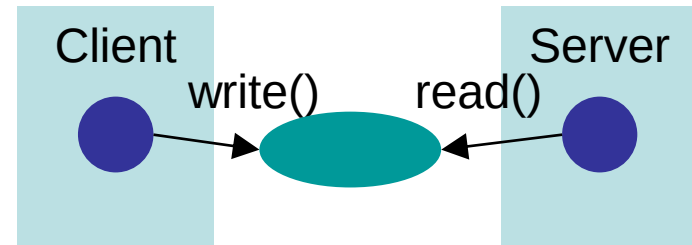
- ▶ Persistency
- ▶ Security
- ▶ Naming
- ▶ Events
- ▶ Transaction
- ▶ Trader
- ▶ Accounting

# Models

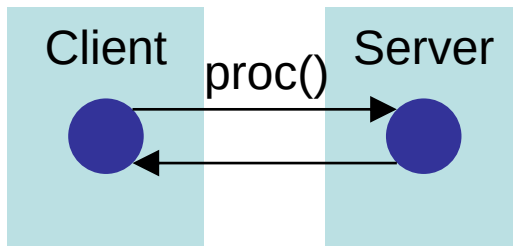
Message passing



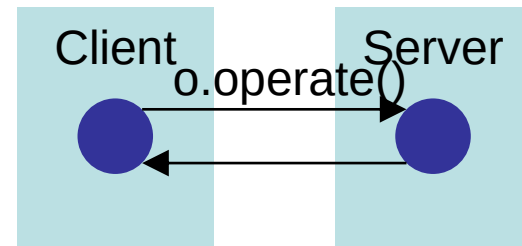
Virtual shared memory



Remote procedure call



Distributed object systems



# Examples

- ▶ Remote procedure call
  - ▶ DCE RPC
- ▶ Distributed object system
  - ▶ OMG CORBA
- ▶ Message passing
  - ▶ IBM MQSeries
- ▶ Virtual shared memory
  - ▶ Linda

# Remote procedure call (RPC)

# Online shop example

- ▶ Implementing an online shop client and server with just sockets is tedious
- ▶ Smart approach:
  - ▶ `public Receipt order(Book[] books)`
  - ▶ `public Book[] search(String keyword)`
- ▶ Client calls procedures on the server as if they were local procedures -> RPC

# RPC – Overall Goal 1/2

- ▶ Provide distribution transparency
  - ▶ Programming as if there is no distribution
  - ▶ “Hunt for the holy grail”
  - ▶ We will get quite close, but we will never get hold of the grail
  - ▶ “We argue that objects that interact in a distributed system need to be dealt with in ways that are intrinsically different from objects that interact in a single address space. These differences are required because distributed systems require that the programmer be aware of latency, have a different model of memory access, and take into account issues of concurrency and partial failure.” [1]
  - ▶ Degenerated case: Programming as if there is only distribution

[1] S. C. Kendall, J. Waldo, A. Wollrath and G. Wyant: A Note on Distributed Computing

# RPC – Overall Goal 2/2

- ▶ Hide expert knowledge in the tool chain
  - ▶ “Sockets”
  - ▶ Session management
  - ▶ Data representation
  - ▶ Data transformation
  - ▶ Service discovery
  - ▶ ... you name it ...

# Remote procedure call

- ▶ RPC sits on top of the transport layer (presentation/session layers)
  - ▶ Hiding network communication from application programmer, i.e., building abstraction
  - ▶ Sockets etc. are not visible to the application programmer
- ▶ Usually a request-reply protocol is specified
  - ▶ Procedure invocation message (Request)
  - ▶ Procedure result message (Reply)
- ▶ RPC is responsible for *Per e' invio di strutture dati*
  - ▶ Marshalling & unmarshaling data (parameters and results)
  - ▶ External data representation
  - ▶ Addressing

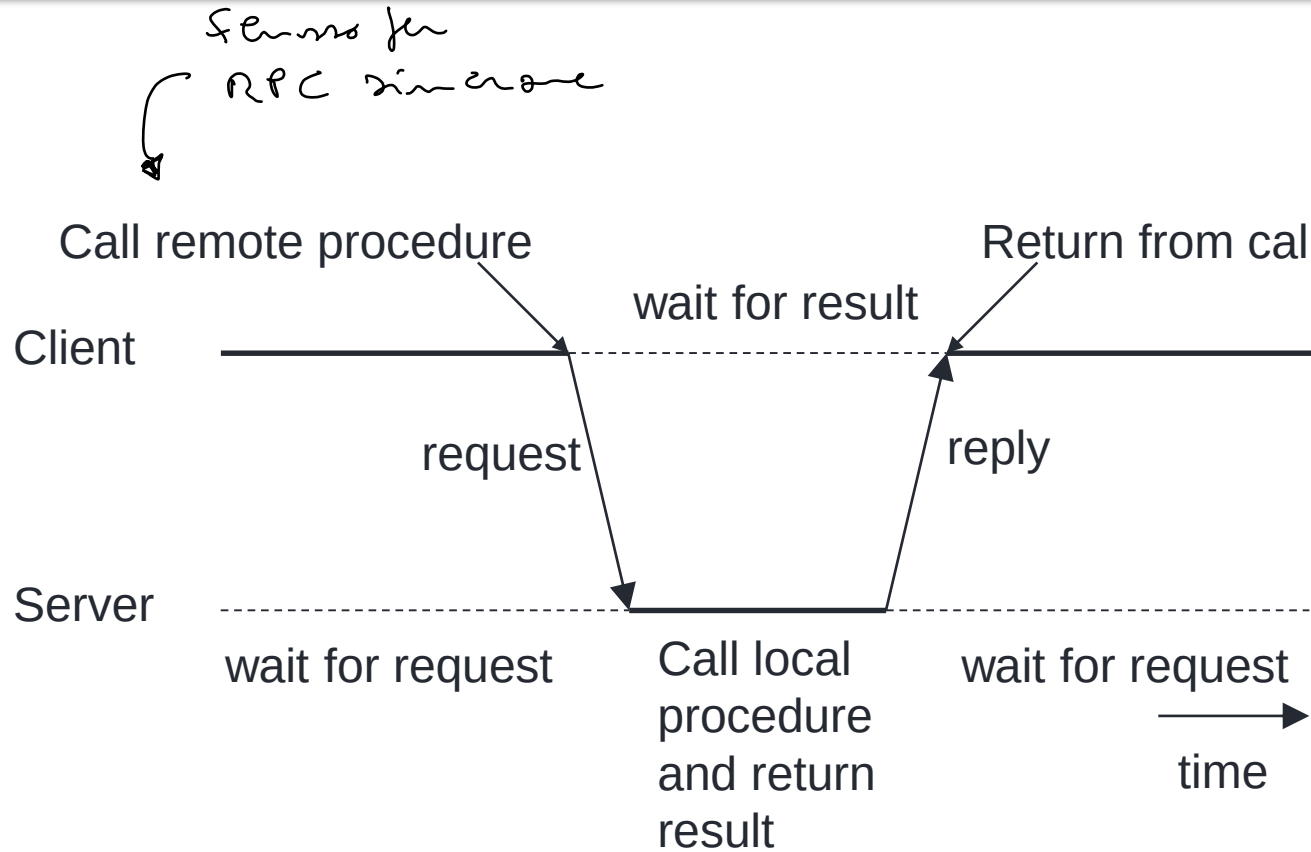
# Problems

- ▶ Heterogeneity
  - ▶ Different data representations (little-endian vs. big-endian, ASCII vs. EBCDIC)
- ▶ Addressing
  - ▶ How to identify a remote process?
- ▶ Partial failure → *Occorre conoscere le semantics del fallimento*
  - ▶ What happens if the server crashes during execution?
    - ▶ Can we guarantee “at least once” semantics?
  - ▶ What happens if the client crashes?
    - ▶ Can we detect and remove orphans?
  - ▶ What happens if a crashed machine is rebooted?
    - ▶ Can addresses survive a reboot?

# How is it implemented?

- ▶ **Client-side proxy (stub)**
  - ▶ Implements the interface of the target procedure on the client side
  - ▶ Client calls this interface **locally** (-> transparency)
  - ▶ Procedure is invoked on a **remote** machine
- ▶ **Server-side proxy (stub or skeleton)**
  - ▶ Implements the interface of the target procedure on the server side
  - ▶ Incoming requests are dispatched to this interface **locally**
    - ▶ Server does not realize that the call is a **remote** call
- ▶ The result is: almost distribution transparency ...
  - ▶ ... if there is no failure

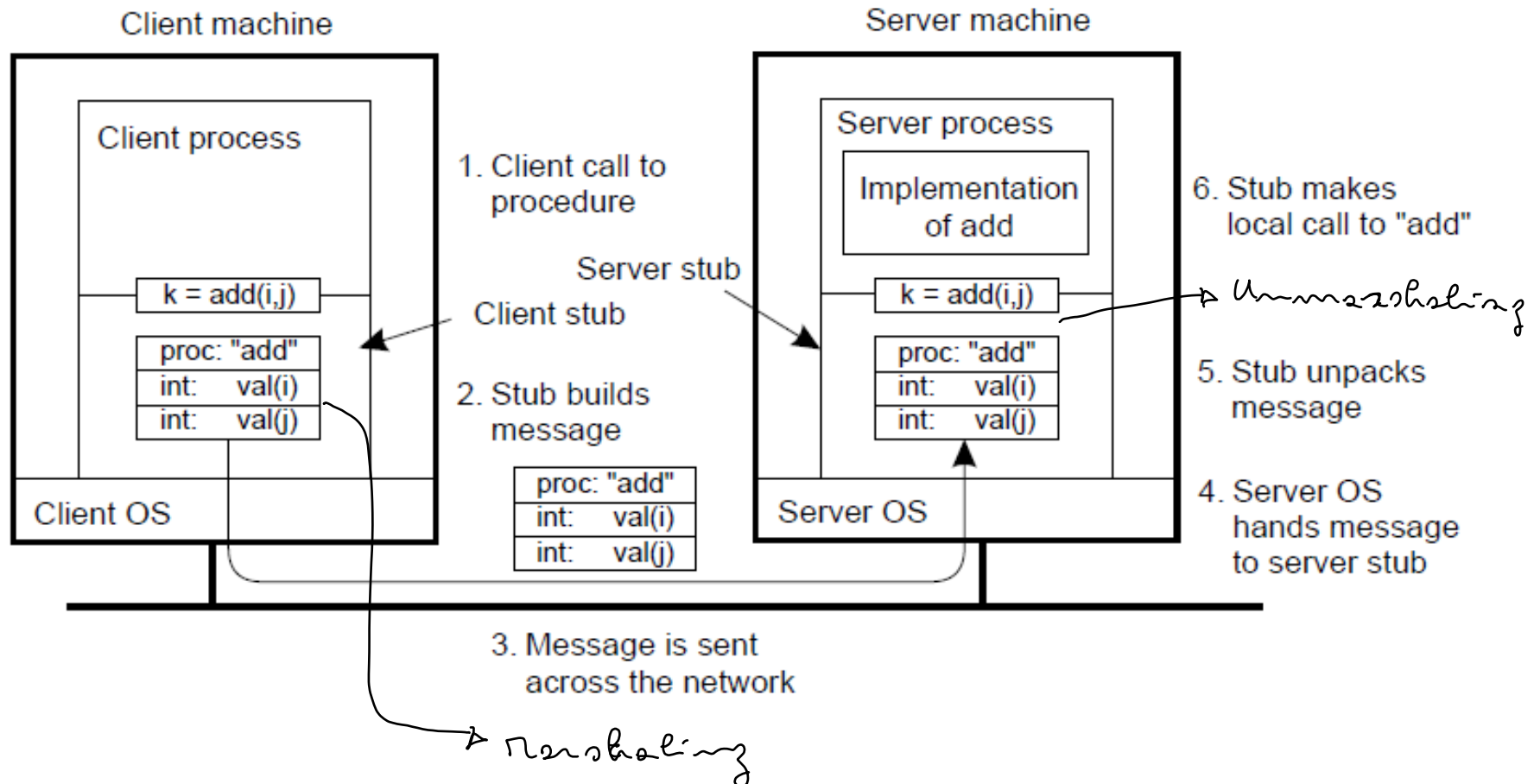
# Flow of action



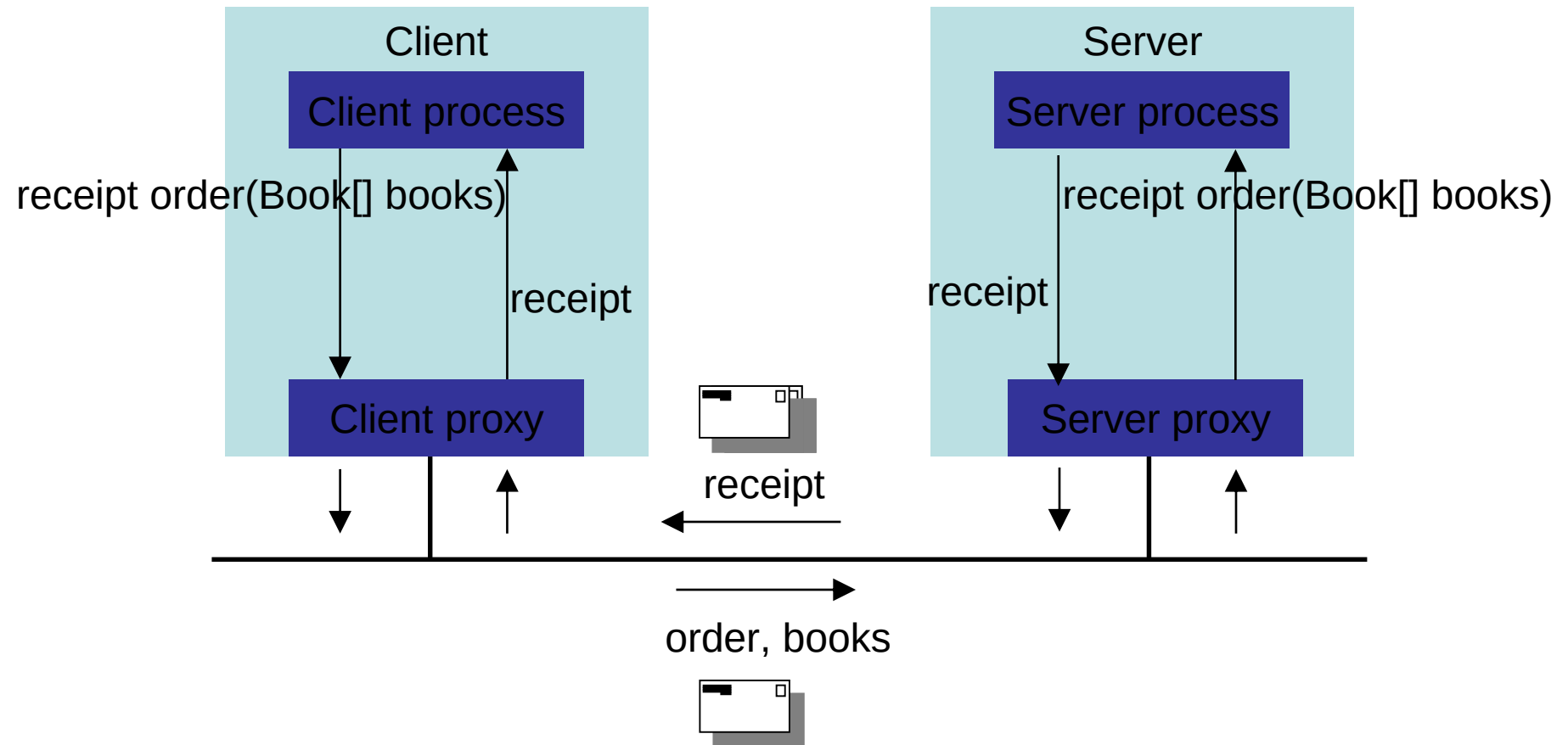
# Flow of action

1. Client calls procedure on client-side proxy locally
2. Client proxy marshals parameters and sends message to server proxy
3. Server proxy unmarshals parameters and calls procedure locally
4. Procedure does work and returns result to the proxy
5. Server proxy marshals result and sends message to client proxy
6. Client proxy unmarshals result and returns to client

# Request flow



# Bookstore example



# Calling semantics

- ▶ Local case
  - ▶ Call-by-value
  - ▶ Call-by-reference
- ▶ Call-by-reference remotely is difficult
  - ▶ Client and server do not share an address space
  - ▶ Simulate by call-by-copy/restore
    - ▶ Transmit copy of the data and transmit changed copy back
    - ▶ Slightly different semantics!
  - ▶ Increased overhead for collections (graphs, lists,...)

# Marshalling / Unmarshalling

- ▶ Assembling to external format: Marshalling
- ▶ Disassembling from external format: Unmarshalling
- ▶ Complex data types must be serialized
  - ▶ Lists, Structs, Graphs, ...
- ▶ When data formats of machines differ
  - ▶ Use agreed common external format
    - ▶ Sender transforms to external
    - ▶ Receiver transforms external to his own
  - ▶ Use sender's format and "receiver makes it right"
    - ▶ Sender must send indication of format

# Proxy Generation

- ▶ We need a **tool** that generates the proxies
- ▶ The tool has no knowledge about
  - ▶ The interface
    - ▶ Procedure names
    - ▶ Parameter types
    - ▶ Return types
    - ▶ Exceptions
  - ▶ Data types

# Interface definition

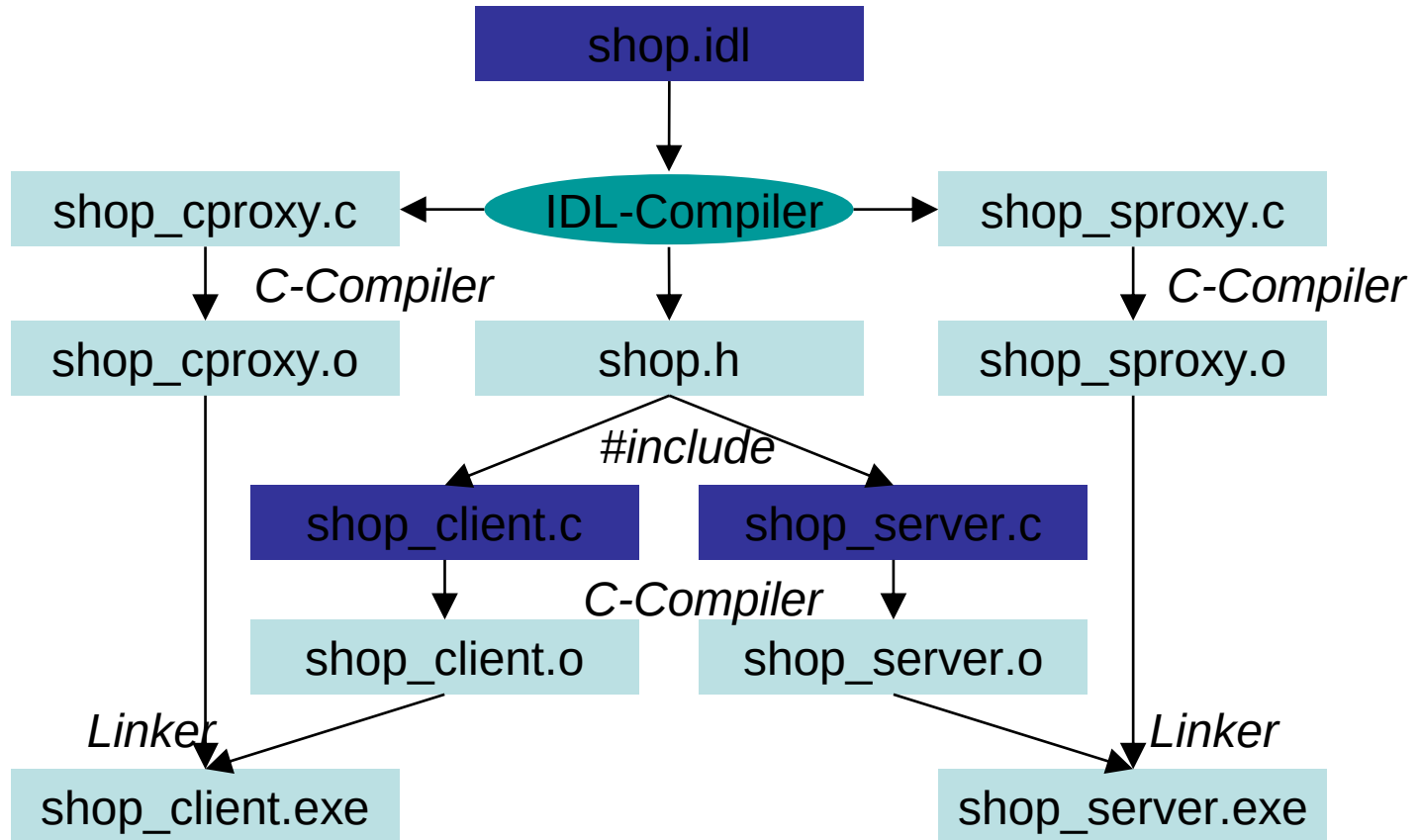
- ▶ Interfaces are defined by an Interface Definition Language (IDL)
  - ▶ Language-neutral
  - ▶ Usually C-style syntax
- ▶ Proxies can be generated from IDL
  - ▶ Different proxies for different programming languages
  - ▶ E.g. client in Java -> client proxy in Java
  - ▶ E.g. server in C -> server proxy in C

# IDL example

```
module shops{  
    interface bookshop{  
        struct Book{  
            string name;  
            long isbn;  
        };  
        struct Receipt{  
            string bank;  
            long accountnumber;  
            int amount;  
        };  
        Receipt order(in Book[] books);  
        Book[] search(in string keyword);  
    };  
};
```



# IDL in the Tool Chain



# IDL Pros & Cons

## ▶ Pros

- ▶ Language neutral

## ▶ Cons

- ▶ Generated interface can be ugly
  - ▶ Example: CORBA and C++
- ▶ Developers have to master two languages
- ▶ Requires top-down approach
  - ▶ First IDL, then implementation
  - ▶ Cannot simply use existing code & data structures

## ▶ Solutions? Yes!

- ▶ .NET Remoting does not require any IDL

# RPC Failure

- ▶ Local call failure
  - ▶ When call fails, the whole program fails
- ▶ What can go wrong with RPC?
  1. Client cannot find the server
  2. Client crashes after sending request
  3. Request gets lost
  4. Server crashes after receiving request, before sending response
  5. Response gets lost
  6. Client crashes before receiving response

# RPC Failure

- ▶ Fault detection
  - ▶ Wait for expected response
  - ▶ After timeout → failure
- ▶ What is a good timeout?
  - ▶ Maybe the network is too slow
  - ▶ Maybe the other computer is too slow
  - ▶ Usually no real-time ⇒ Calculation of optimal timeout is not possible
- ▶ No way to find out what went wrong
  - ▶ Remote machine does not respond. Why?
    - ▶ Machine crashed?
    - ▶ Message loss?

# Delivery Guarantees

- ▶ **Retry request message**
  - ▶ Client retransmits until it receives a reply or server is assumed to have failed
- ▶ **Duplicate filtering**
  - ▶ Server filters out duplicate messages when retransmissions are used
- ▶ **Retransmission of replies**
  - ▶ A history of replies to retransmit without re-execution

# Client crash

- ▶ Processes on servers for non-existing clients (Orphans)
  - ▶ Block resources
- ▶ Solution
  - ▶ Client sends “heartbeat” (e.g., to obtain stat info)
  - ▶ Server pings client
  - ▶ Pings & heartbeats are expensive and subject to failures, too
- ▶ Client restart
  - ▶ Do not mix new with old messages
  - ▶ E.g. counter for every restart

# Server crash

- ▶ Client gets a timeout waiting for the response
  - ▶ Did the server process the request?
- ▶ Possible semantics
  - ▶ Exactly-once
    - ▶ Transactions (only if with persistent memory)
  - ▶ Maybe *→ non affidabile*
    - ▶ Nothing to be done
  - ▶ At-least-once
    - ▶ Repeat until response received
  - ▶ At-most-once
    - ▶ Serial numbers for requests, repeat means failure

# Server crash

- ▶ **“exactly-once”** call semantics
  - ▶ The **most desirable** kind of semantics
  - ▶ Every call is carried out exactly once, no more and no less
  - ▶ Such semantics **cannot be achieved at low cost**
    - ▶ If the client transmits a request, and the server crashes, the client has no way of knowing whether the server had received and processed the request before crashing

# Server crash

## ▶ “maybe” call semantics

- ▶ No retransmission of request messages
- ▶ Not certain whether the procedure has been executed
- ▶ No fault-tolerance measures
- ▶ Generally not acceptable

# Server crash

- ▶ **“at-least-once”** call semantics
  - ▶ The msg module repeatedly resends request msg after timeouts until it either gets a reply msg or some max. # of retries have been made
  - ▶ No duplicate request msg filtering
  - ▶ The remote procedure is called at least once if server not down
  - ▶ The client does not know how many times the remote procedure has been called
  - ▶ Unless the called procedure is **“idempotent”** (i.e., repeatable), this could produce undesirable results (e.g., money transfer)
  - ▶ If the server crashes no way of knowing whether the server has received and processed the request

# Server crash

- ▶ **“at-most-once”** call semantics
  - ▶ Retransmission of request messages
  - ▶ Duplicate request msg filtering
  - ▶ If the server does not crash and the client receives the result of a call, then the procedure has been called exactly once
  - ▶ Otherwise, an exception is reported and the procedure will have been called once or not at all
  - ▶ This works for both idempotent and non-idempotent operations
  - ▶ More complex support required due to the need for request msg filtering and for keeping track of replies

# Call Semantics

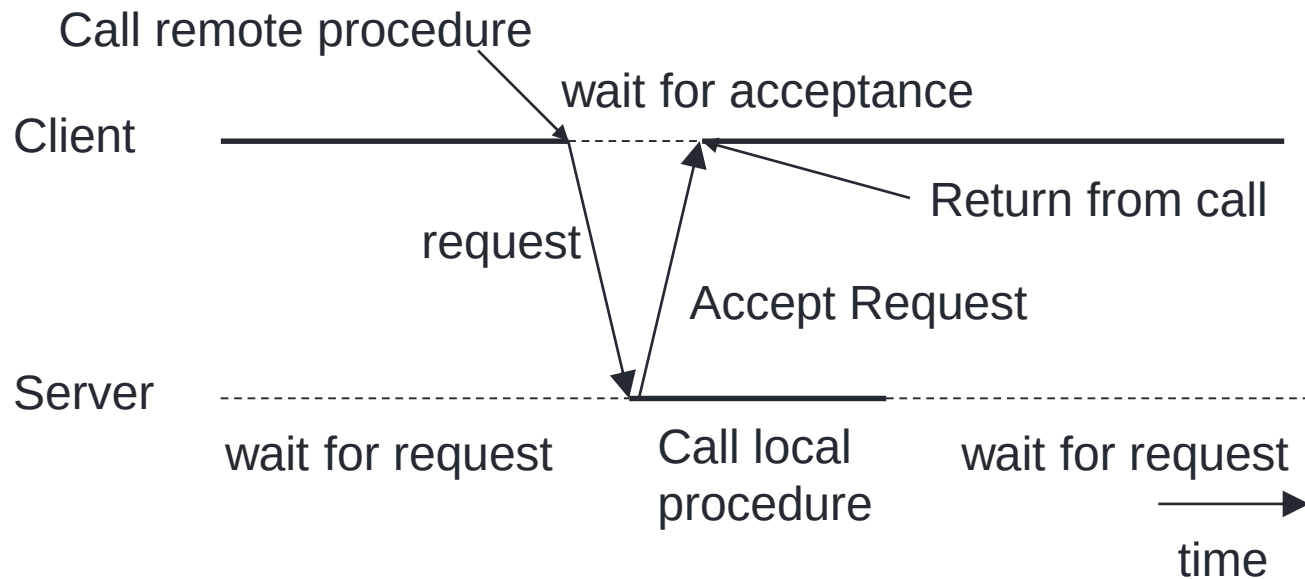
| RPC call semantics | Retry request message | Duplicate filtering | Response             |
|--------------------|-----------------------|---------------------|----------------------|
| Maybe              | No                    | Not Applicable      | Not Applicable       |
| At-least-once      | Yes                   | No                  | Re-execute procedure |
| At-most-once       | Yes                   | Yes                 | Retransmit reply     |



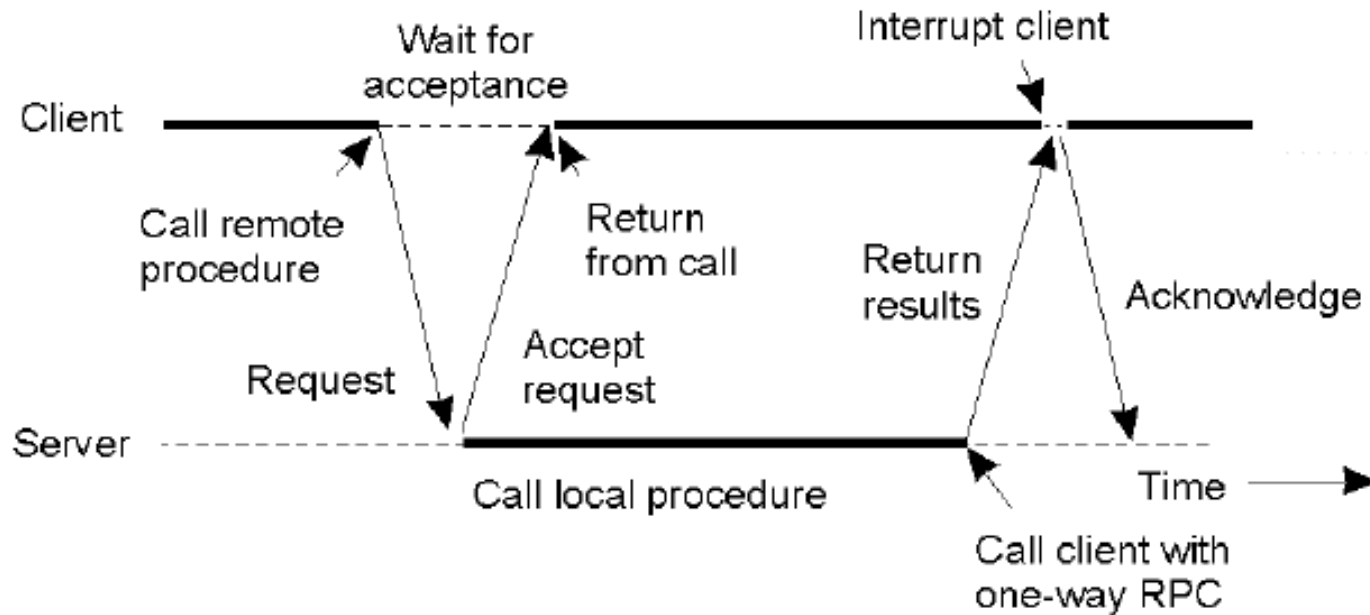
# RPC call variants

- ▶ **Synchronous (blocking) call**
  - ▶ Parallelism in distributed system is not exploited
  - ▶ No parallel invocations to multiple servers
- ▶ **Asynchronous RPC**
  - ▶ “Fire-and-forget”
  - ▶ Wait for acceptance and continue
  - ▶ Server can reply as soon as request is received and execute procedure later
  - ▶ Useful when there is no result to return
- ▶ **Deferred synchronous RPC**
  - ▶ Do something while server is executing
  - ▶ Use two asynchronous RPCs
  - ▶ Client needs a reply but cannot wait for it; server sends reply via another asynchronous RPC

# Asynchronous RPC



# Deferred Synchronous RPC



# Example RPC systems

- ▶ Sun RPC
  - ▶ Used by the Network File System (NFS)
- ▶ Distributed Computing Environment (DCE) RPC
  - ▶ Basis for Microsoft's DCOM (Component Object Model)
  - ▶ RPC (i.e. DCOM) has been used in several exploits

# Other Solutions: Distributed objects

- ▶ Today's commonly used programming languages are object-oriented
- ▶ “*Remote objects*”
  - ▶ Objects that can receive method invocations from objects in other processes
  - ▶ Including processes on a different machine
- ▶ Objects get a *remote interface*
  - ▶ Usually defined in IDL
- ▶ Client needs a *remote object reference* to perform a *remote method invocation*
- ▶ *Distribute objects similar to RPC but suitable for objects*

# CORBA

- ▶ Common Object Request Broker Architecture
- ▶ Standard of the Object Management Group (OMG) - [www.omg.org](http://www.omg.org)
  - ▶ Not a specific system
  - ▶ Describes a whole software architecture (OMA)
- ▶ Programming language-independent
  - ▶ CORBA IDL to define object interfaces
- ▶ Platform-independent
  - ▶ Object Request Brokers (ORBs) interoperate via specified protocols
  - ▶ GIOP (General Inter-ORB Protocol) & IIOP (Internet Inter-ORB Protocol)
- ▶ Complete distributed object infrastructure
  - ▶ Lots of additional services

# .Net Remoting

- ▶ Distributed objects in .Net
- ▶ Language independent
  - ▶ As long as it's a .Net language
  - ▶ No IDL
- ▶ Highly configurable
  - ▶ Can be integrated with other systems

# Java RMI

- ▶ Remote Method Invocation
  - ▶ Java implementation of RPC
  - ▶ Allow to call a remote method on a remote object

# Conclusions

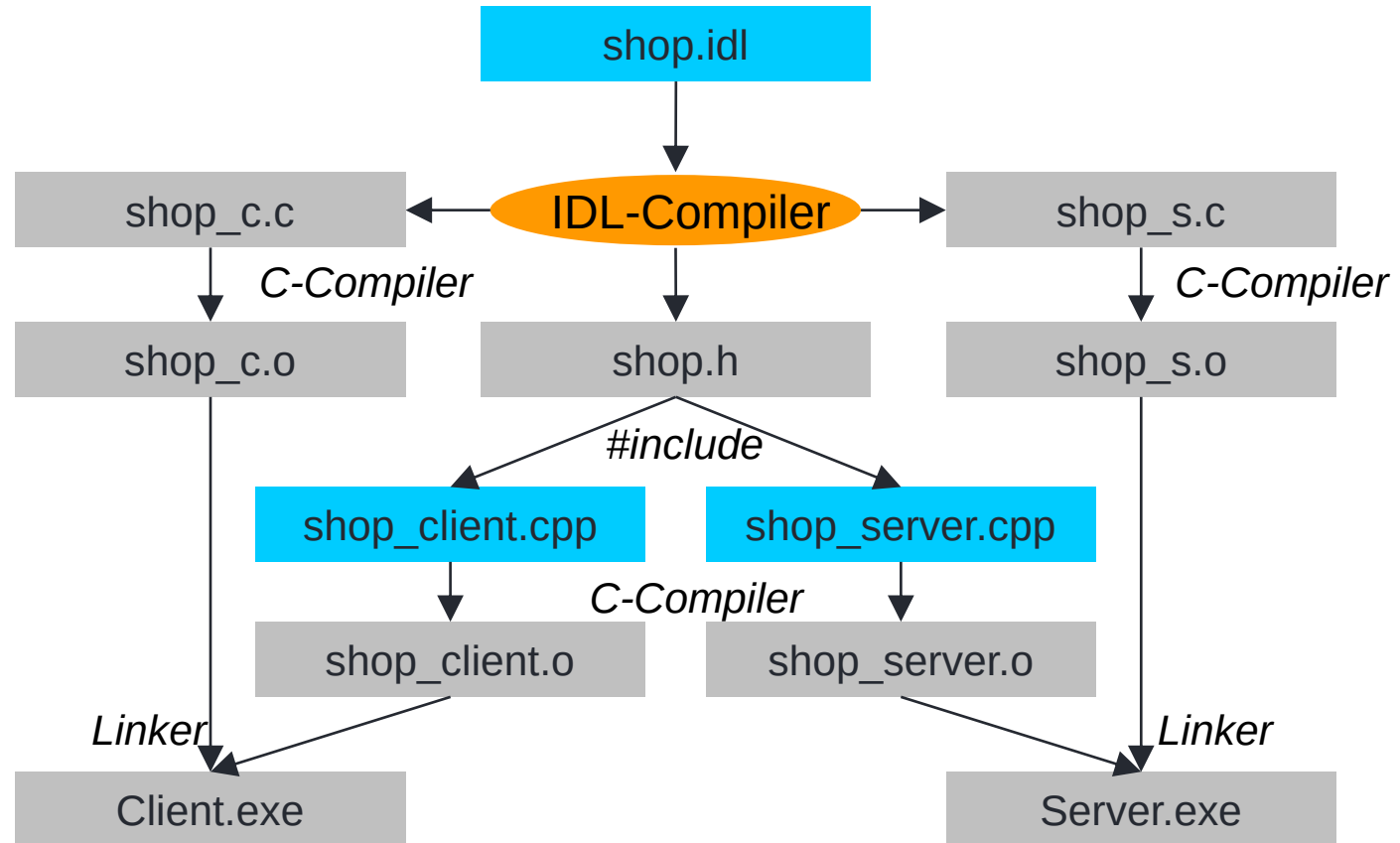
- ▶ Socket programming is too low level
- ▶ Abstractions can greatly simplify remote communications
  - ▶ Middleware
  - ▶ RPC: Client calls procedures on the server as if they were local procedures
  - ▶ Provide additional services
- ▶ Problems: endpoints crashes, remote objects



# RPC: Caso di Studio



# Descrizione Struttura del progetto



# File shop.IDL

**Interface  
Header**

```
[ uuid {906BOCEO-C70B-1067-B317-00DD010662DA},  
  version(1.0)  
]
```

**Interface  
Body**

```
interface bookshop{  
  
    struct Book{  
        [string] unsigned char* name;  
        long isbn;  
    };  
  
    struct Receipt{  
        [string] unsigned char* bank;  
        long accountnumber;  
        int amount;  
    };  
  
    struct Receipt order([in] struct Book book);  
    struct Book search([in,string] unsigned char* keyword);  
};
```



# File shop.IDL: Interface Header

```
[ uuid {906B0CEO-C70B-1067-B317-00DD010662DA} ,  
  version(1.0)  
]
```

- ▶ Contiene informazioni generali relative all'interfaccia
- ▶ Gli attributi definiti sono globali rispetto a tutta l'interfaccia
- ▶ *UUID*: assegna all'interfaccia un Universally Unique Identifier che la identifica univocamente
- ▶ *Version*: definisce la versione dell'interfaccia

# File shop.IDL: Interface Body (1)

|                                      |   |                                                |
|--------------------------------------|---|------------------------------------------------|
| Nome interfaccia                     | { | <code>interface bookshop{</code>               |
|                                      |   |                                                |
| Definizione struttura <i>Book</i>    | { | <code>struct Book{</code>                      |
|                                      |   | <code>    [string] unsigned char* name;</code> |
|                                      |   | <code>    long isbn;</code>                    |
|                                      |   | <code>};</code>                                |
|                                      |   |                                                |
| Definizione struttura <i>Receipt</i> | { | <code>struct Receipt{</code>                   |
|                                      |   | <code>    [string] unsigned char* bank;</code> |
|                                      |   | <code>    long accountnumber;</code>           |
|                                      |   | <code>    int amount;</code>                   |
|                                      |   | <code>};</code>                                |

- **[string]** indica di trattare il campo (*unsigned char\**) come un campo *string*

# File shop.IDL: Interface Body (2)

```
struct Receipt order([in] struct Book book);  
struct Book search([in,string] unsigned char* keyword);
```

- ▶ Definizione funzioni remote chiamate dal client ed esposte dal server
  - ▶ L'attributo **[in]** indica che il parametro è passato dalla procedura chiamante alla procedura chiamata → passaggio per valore
  - ▶ L'attributo **[out]** indica che il puntatore al parametro deve essere restituito dal server al client. Deve essere sempre seguito da una variabile puntatore → Per riferimento
  - ▶ E' permesso indicarli contemporaneamente: **[in,out]**

# File shop.ACF

```
[implicit_handle(handle_t server_Handle)]  
  
interface bookshop  
{  
  
}
```

- ▶ Contiene configurazioni specifiche dell'implementazione locale e non dell'interfaccia di rete (hardware e sistema operativo)
- ▶ **implicit\_handle** implicitamente indica il nome e tipo del parametro *server\_Handle*, che rappresenta il server, per evitare di indicarlo in tutti i metodi remoti
- ▶ I file ACF e IDL sono necessari al compilatore MIDL (Microsoft) per generare i proxy

# IDL compiler Output

- ▶ MIDL richiede i file `shop.IDL` e `shop.ACF` e genera i file:
  - ▶ `shop.h` (o `shop_h.h`): header file da includere nei sorgenti del client e del server; contiene le signature dei metodi remoti
  - ▶ `shop_c.c`: client proxy, implementa i metodi remoti indicati nell'IDL e richiamati dal client; gestisce la comunicazione con il server
  - ▶ `shop_s.c`: server proxy, gestisce la comunicazione con i client e l'esposizione dei metodi remoti
- ▶ `shop.h` deve essere incluso nel codice del client e del server
- ▶ `shop_c.c` e `shop_s.c` devono essere linkati, rispettivamente, con il client e il server in fase di compilazione

# Applicazione Client

```
#include <stdlib.h>
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#include "../IDL/shop.h"
```

- Il file *shop.h* creato da MIDL deve essere importato con il percorso corretto

# Applicazione Client

```
//Connection status
RPC_STATUS status;

//universally unique identifier of the interface
unsigned char * pszUuid          = NULL;

//underlying network transport protocol
unsigned char * pszProtocolSequence = (unsigned char*)"ncacn_np";

//network address (NULL for local address)
unsigned char * pszNetworkAddress  = NULL;
```

- ▶ *status*: stato attuale della connessione
- ▶ *pszUuid*: UUID dell'interfaccia connessa (*NULL* perchè unica)
- ▶ *pszProtocolSequence*: protocollo di rete utilizzato (*ncacn\_np* per le comunicazioni tra processi)
- ▶ *pszNetworkAddress*: indirizzo di rete della macchina (*NULL* per indirizzo locale)

# Applicazione Client

```
//interface to use
unsigned char * pszEndpoint      = (unsigned char*)"\\pipe\\bookshop";

//null-terminated string representation of network options
unsigned char * pszOptions      = NULL;

//represent the binding object (protocol, network address, endpoint)
unsigned char * pszStringBinding = NULL;

//exception code
unsigned long ulCode;
```

- ▶ *pszEndpoint*: indica che canale utilizzare; la sintassi dipende dal protocollo specificato
- ▶ *pszOptions*: opzioni generali di rete
- ▶ *pszStringBinding*: stringa che rappresenta la connessione da stabilire (protocollo, indirizzo ed endpoint)

# Applicazione Client

```
//creates a string binding handle

status = RpcStringBindingCompose(pszUuid,pszProtocolSequence,
                                pszNetworkAddress,pszEndpoint,
                                pszOptions,&pszStringBinding);

if (status){
    exit(status);
}
```

- ▶ *RpcStringBindingCompose* crea una stringa di connessione per la gestione della comunicazione
- ▶ Restituisce: *RPC\_S\_INVALID\_STRING\_UUID* se ci sono errori, *RPC\_S\_OK* se tutto ok
- ▶ Valorizza la variabile *pszStringBinding* che contiene la stringa di connessione da utilizzare per stabilire la connessione

# Applicazione Client

```
//returns a binding handle from a string representation of a binding object
status = RpcBindingFromStringBinding(pszStringBinding,
                                     &server_Handle); //server binding handle

if (status){
    exit(status);
}
```

- ▶ Stabilisce la connessione tra client e server a partire da *pszStringBinding* creato precedentemente
- ▶ Restituisce lo stato della connessione ed eventuali errori
- ▶ Valorizza *server\_Handle* che contiene il descrittore (handle) della connessione stabilita

# Applicazione Client

```
//macros that control exceptions that occur outside the application code
RpcTryExcept
{
    bookToBuy = search(title);
    printf("Book found : \n");
    printf("\t Title: %s \n", bookToBuy.name);
    printf("\t ISBN: %li \n",bookToBuy.isbn);

    receipt = order(bookToBuy);
    printf("\n\nReceipt received : \n");
    printf("\t Bank: %s \n", receipt.bank);
    printf("\t Account Number: %li \n",receipt.accountnumber);
    printf("\t Amount: %i \n",receipt.amount);
}
RpcExcept(1) //if some exceptions occurs
{
    ulCode = RpcExceptionCode(); //retrives a code that identifies the type of exception that occurred
    printf("Runtime reported exception 0x%lx = %ld\n", ulCode, ulCode);
}
RpcEndExcept
```

# Applicazione Client

- ▶ Macro *RpcTryExcept* e *RpcExcept* forniscono un meccanismo per la gestione di eccezioni in sistemi RPC
- ▶ Se nel blocco *RpcTryExcept* si riscontra un'eccezione causata da una procedura remota, il controllo passa al blocco *RpcExcept* per la gestione della stessa
- ▶ *RpcExceptionCode()* restituisce il codice dell'eccezione, utilizzabile per eventuali messaggi d'errore
- ▶ *RpcEndExcept* chiude il blocco *RpcTryExcept*
- ▶ Utilizzabile solo sotto sistemi Microsoft, **NON** è standard ANSI C

# Applicazione Client

- ▶ Al termine della connessione, *RpcBindingFree* libera la memoria utilizzata da *server\_Handle*

```
//release the handle to the server
status = RpcBindingFree(&server_Handle);
```

- ▶ Funzioni da implementare obbligatoriamente per permettere al sistema l'allocazione e la deallocazione della memoria

```
//Allocate memory before procedure calls
void __RPC_FAR * __RPC_USER MIDL_user_allocate(size_t len)
{
    return(malloc(len));
}

//free memory after procedure calls
void __RPC_USER MIDL_user_free(void __RPC_FAR * ptr)
{
    free(ptr);
}
```



# Applicazione Server

```
//Connection status
RPC_STATUS status;

//underlying network transport protocol
unsigned char * pszProtocolSequence = (unsigned char*)"ncacn_np";

//optional parameter provided for the security subsystem (always NULL)
unsigned char * pszSecurity = NULL;

//endpoint-address information
unsigned char * pszEndpoint = (unsigned char*)"\\pipe\\bookshop";
```

- ▶ *pszSecurity*: parametro opzionale per la gestione della sicurezza, ignorato da quasi tutti i protocolli di rete; da impostare a NULL

# Applicazione Server

```
//minimum number of calls threads
unsigned int    cMinCalls      = 1;

//maximum queue length
unsigned int    cMaxCalls      = 20;

//define RpcServerListen behaviour
unsigned int    fDontWait      = FALSE;
```

- ▶ *cMinCalls*: numero minimo di richieste di connessione da attendere prima di aprire i canali; `RPC_C_LISTEN_MAX_CALLS_DEFAULT` indica il valore imposto di default dal sistema
- ▶ *cMaxCalls*: numero massimo di connessioni gestite contemporaneamente; `RPC_C_PROTSEQ_MAX_REQS_DEFAULT` indica il valore imposto di default dal sistema
- ▶ *fDontWait*: = 0 (*FALSE*) il server attende la conclusione di una connessione per continuare (*bloccante*),  $\neq 0$  (*TRUE*) il server continua l'elaborazione anche durante una connessione (*non bloccante*)

# Applicazione Server

```
//use the specified protocol sequence combined with the specified
//endpoint for receiving remote procedure calls
status = RpcServerUseProtseqEp(pszProtocolSequence,cMaxCalls,
                               pszEndpoint,pszSecurity);

if (status){
    exit(status);
}
```

- ▶ *RpcServerUseProtseqEp* indica al sistema di utilizzare il protocollo specificato, combinato con l'entry point, per gestire l'invocazione di procedure remote, riferite a quel protocollo
- ▶ Restituisce lo stato della connessione ed eventuali errori
- ▶ Un server può registrare più protocolli per gestire diversi canali di comunicazione

# Applicazione Server

```
//registers the interface with the RPC run-time library
status = RpcServerRegisterIf(bookshop_v1_0_s_ifspec, //interface to register
                             NULL,
                             NULL);
```

- ▶ *RpcServerRegisterIf* registra l'interfaccia nel sistema RPC; ogni richiesta verso quell'interfaccia sul protocollo di rete specificato sarà reindirizzata verso l'applicazione server
- ▶ Accetta come parametri l'interfaccia, l'UUID dell'interfaccia e un puntatore ad un vettore di entry point
- ▶ *bookshop\_v1\_0\_s\_ifspec*: oggetto creato dal compilatore MIDL che identifica l'interfaccia *bookshop*
- ▶ MIDL utilizza la seguente sintassi per identificare le interfacce  
$$\langle \text{nome} \rangle\_v\langle \text{major number} \rangle\_ \langle \text{minor number} \rangle\_ \langle s | c \rangle\_ \text{ifspec}$$
- ▶ Restituisce lo stato della connessione

# Applicazione Server

```
//listen for remote procedure calls
status = RpcServerListen(cMinCalls, //minimum number of call threads
                        cMaxCalls, //maximum number of concurrent remote procedure calls
                        fDontWait); //define RpcServerListen behaviour
```

- ▶ *RpcServerListen* indica al sistema RPC di rimanere in attesa di chiamate a procedure remote
- ▶ Accetta come parametri il valore minimo e massimo di connessioni e se la comunicazione deve essere bloccante o non bloccante
- ▶ Restituisce lo stato della comunicazione dopo la chiamata a *RpcServerListen*
- ▶ *RpcServerListen* rimane in esecuzione fino a che il server non viene terminato

# Applicazione Server

```
//search a book in the library
struct Book search(unsigned char* keyword){

    .....

}

//order a book to the library
struct Receipt order(struct Book book){

    .....

}
```

- ▶ I metodi remoti vanno gestiti come funzioni normali, seguendo però la signature definita nel file IDL
- ▶ Le strutture definite nel file IDL sono visibili in tutta l'applicazione
- ▶ La gestione della comunicazione verso i client è a carico della classe *shop\_s.c* generata dal compilatore MIDL, da linkare in fase di compilazione

# Applicazione Server

```
//Allocate memory before procedure calls
void __RPC_FAR * __RPC_USER MIDL_user_allocate(size_t len)
{
    return(malloc(len));
}

//free memory after procedure calls
void __RPC_USER MIDL_user_free(void __RPC_FAR * ptr)
{
    free(ptr);
}
```

- Le funzioni per l'allocazione e la deallocazione della memoria devono essere codificate anche nel server

